The 262-operation $\langle 4 \times 4 \times 4 : 48 \rangle$ algorithm is also the more accurate one: error-bound exponent 9/4 and about a quarter less error

Greg Sidebottom

Draft — November 1, 2026

Abstract

Dumas, Pernet and Sedoglavic's accurate rational $\langle 4 \times 4 \times 4 : 48 \rangle$ algorithm (Table 1 of their 2026 paper; `4x4x4_48_accurate` in PLinOpt) has $(\infty, 2)$-growth factor $\gamma_{\infty,2} = 18\sqrt{2} \approx 25.46$, an error-bound exponent $\log_4 \gamma_{\infty,2} \approx 2.335$. Their later 262-operation variant `4x4x4_48_accurate_262`, in the same orbit, was published for its lower operation count. We show that it is also the more accurate algorithm. Its growth factors are smaller in all four norms they consider: $\gamma_{\infty,2} = 16\sqrt{2}$, an exponent of exactly $9/4$; $\gamma_{\infty,\infty} = 132$ against 148; $\gamma_{2,\infty} = 16\sqrt{963}$ against $8\sqrt{4142}$; and $\gamma_{2,2} = 64\sqrt{2}$ against $8\sqrt{130}$. Its $\gamma_2 = 64(1+\sqrt{2})$ is the same, and the combinatorial factor Q_0 of their error bounds is 34 against 46. Measured against exact references on random matrices, in single and double precision at one to three recursion levels, its error is 0.75–0.77 of `4x4x4_48_accurate`'s on average (0.62–0.90 for single instances), at the same speed in a fused-pass implementation over Apple's Accelerate. In recursion down to scalars its advantage grows, to 0.72 at five levels. We also minimize $\gamma_{\infty,2}$ over the whole de Groote orbit of the scheme and its three role assignments, with an exact-gradient search. The orbit's minimum is $\gamma_{\infty,2} = 21.41$, exponent 2.210, reached in the two other role assignments at points whose coefficients lie in $\mathbb{Q}(\sqrt{2})$; in the published role the minimum is $16\sqrt{2}$, at points equivalent to `4x4x4_48_accurate_262`, which we found before we knew of it. Among the orbit's 648 dyadic orthogonal points, enumerated exhaustively, the points minimizing both $\gamma_{\infty,2}$ and $\gamma_{\infty,\infty}$ are all equivalent to `4x4x4_48_accurate_262`. The 21.41 point cannot be made dyadic, and in floating point, where its constants round, its measured error is no lower than `4x4x4_48_accurate`'s. Dyadic probe points show that neither $\gamma_{\infty,2}$ nor $\gamma_{\infty,\infty}$ alone predicts the measured error, while Q_0 and exact dyadic coefficients matter. A first-order rounding model of the programs, taken as a search objective, gives programs with 10% less modelled rounding error that measure as a tie, because the leaf products' error, amplified by the scheme, dominates; that amplification, $2\sqrt{2}$ per level, is the minimum over the orbit, over the other known rank-48 classes and over the variety of decompositions around them. Adding in double precision inside the single-precision passes, at no cost below $n = 16384$, cuts the error at five recursion levels by a third. The other known rational classes, Li, Wang and Hu's curve and Agarwal's scheme, have larger $\gamma_{\infty,2}$ minima. Finally, a search seeded with DPS's programs computes the same matrices in 260 operations instead of 262, with the same growth factors, at the same speed, and with measured error within about 2% at one to three levels.

1 Introduction

Two 4×4 matrices can be multiplied with 48 multiplications over any ring containing $1/2$. AlphaEvolve found a rank-48 scheme with complex coefficients [19], and Dumas, Pernet and Sedoglavic

(DPS) turned it into one with rational coefficients [10]. Applied recursively, such a scheme multiplies $n \times n$ matrices with $O(n^{\log_4 48}) = O(n^{2.7925})$ operations. Fast schemes trade accuracy for speed, and their error bounds are governed by a *growth factor* computed from the scheme's coefficient matrices [4, 3, 8, 15]. DPS generalized it to a (p, q)-growth factor $\gamma_{p,q}$ for any output and input norms [9, 12], and they argue that the $(\infty, 2)$-factor $\gamma_{\infty,2}$ best reflects the accuracy observed in practice.

In [11] DPS proposed an accurate variant of their rational scheme, given as Table 1 of that paper and in PLinOpt as `4x4x4_48_accurate` [14]. They found it in the de Groote isotropy orbit [7] of their first scheme: a point minimizing the smooth bound $\gamma_2 \geq \gamma_{2,2}$ along the orbit, then moved by orthogonal isotropies, which keep γ_2, to get sparse straight-line programs. Its growth factor is $\gamma_{\infty,2} \approx 25.456$, error-bound exponent $\log_4 \gamma_{\infty,2} \approx 2.335$, the smallest among the algorithms they compare, and its programs take 284 operations. The paper's versions run from 19 March 2026 (v1) to 29 July 2026 (v3, the current one), and `4x4x4_48_accurate` entered PLinOpt on 23 June 2026.

A cheaper accurate variant, with 262 operations, came next. Its earliest trace is the second version of [13], dated 2 October 2026, whose introduction states, without a reference or coefficients, that the best currently known accurate variant has leading term 9+3/16, or 5+3/4 in an alternative basis. The leading term is $1 + q/32$ for q additions, subtractions and scalings, so $9 + 3/16$ means $q = 262$. The first version, of 10 September 2026, mentions only `4x4x4_48_accurate` (leading term 9.875). The scheme itself was first published in PLinOpt on 13 October 2026 as `4x4x4_48_accurate_262` (commit `b441528`, by J.-G. Dumas), described as "accurate with 262 operations, or (5+3/4) ALT/CoB". Neither source says how it was found or reports its growth factors, and `4x4x4_48_accurate` remains the accurate variant on record. After this paper's first draft, A. Sedoglavic informed us that the point with $\gamma_{\infty,2} = 16\sqrt{2}$ was found by Touko Ursin in mid-September 2026, by a construction with Hadamard matrices (a reduced form of the parametrization of Section 4) rather than by search, and has not been published yet [22]. In a second message he describes the construction [23]: `4x4x4_48_accurate` is the orbit's γ_2 minimum, unique modulo orthogonal isotropies by their convexity result and found by numerical optimization (pymanopt) over the quotient of GL_4 by the orthogonal group [12, Lemma 27]; Ursin then explored the action of Hadamard matrices, orthogonal isotropies that leave γ_2 unchanged, to lower $\gamma_{\infty,2}$, which gave `4x4x4_48_accurate_262`.

We call schemes by their PLinOpt names: `4x4x4_48_accurate` for Table 1 of [11], `4x4x4_48_accurate_262` for the variant with 262 operations, `4x4x4_48_rational` for the first rational scheme [10], `4x4x4_48_201` for our 201-operation point [24], and `2x2x2_7_Winograd`, `2x2x2_7_Strassen` and `2x2x2_7_DPS-accurate` for the $\langle 2 \times 2 \times 2 : 7 \rangle$ algorithms. Tables drop the common prefixes. Points of our search that are not in PLinOpt have descriptive names. We write `accurate_260` for `4x4x4_48_accurate_262`'s matrices with our 260-operation programs (Section 8.4); it is the same scheme, not a new one.

We show that `4x4x4_48_accurate_262`, the cheaper variant, is also the more accurate one:

- Its growth factors are smaller than `4x4x4_48_accurate`'s in all four norms of [11, Table 2], with the same γ_2: $\gamma_{\infty,2} = 16\sqrt{2}$, an error-bound exponent of exactly 9/4, against 25.456; $\gamma_{\infty,\infty} = 132$ against 148; $\gamma_{2,2} = 64\sqrt{2}$ against 91.21; and $\gamma_{2,\infty} \approx 496.5$ against 514.9. The combinatorial factor Q_0 of DPS's error bounds is 34 against 46 (Section 3).

- Measured on random matrices against exact references, in single and double precision, its error is 0.75–0.77 of `4x4x4_48_accurate`'s on average at one to three recursion levels, and never above 0.90. In recursion down to scalars the advantage grows, to 0.72 at five levels. It runs at the same speed in a fused-pass implementation over Apple's Accelerate (Sections 5 and 6).

Before we knew of `4x4x4_48_accurate_262`, which was first published in PLinOpt (commit `b441528` [14]) and first found by Ursin (Section 1), our search found a scheme equivalent to it, that is, `4x4x4_48_accurate_262` written in another way (Section 2.3). The search minimizes $\gamma_{\infty,2}$ over the orbit and the scheme's role assignments (Section 4). That search, and the value of the orbit's $\gamma_{\infty,2}$ minimum it gives, are the paper's other contribution. In the published role the lowest point the search finds is equivalent to `4x4x4_48_accurate_262`, and it is exact as found: its U and W have entries $0, \pm 1/\sqrt{2}$, which rescaling turns into dyadic coefficients, so no snapping to a dyadic grid is involved. Among the orbit's dyadic orthogonal points, which we enumerate exhaustively, the points minimizing $\gamma_{\infty,\infty}$ also minimize $\gamma_{\infty,2}$, and all of them are equivalent to `4x4x4_48_accurate_262`. The orbit's global $\gamma_{\infty,2}$ minimum, 21.407, is reached only in the other two role assignments and only at points with coefficients in $\mathbb{Q}(\sqrt{2})$, and its measured error is no lower than `4x4x4_48_accurate`'s. The other known rational classes, members of Li, Wang and Hu's curve [18] and Agarwal's scheme [1], have larger $\gamma_{\infty,2}$ minima. So $\gamma_{\infty,2}$ alone is not a sufficient objective for an accuracy search. The combinatorial factor Q_0 and exact dyadic coefficients matter as much. `4x4x4_48_accurate_262` has dyadic coefficients and the smallest Q_0 and $\gamma_{\infty,\infty}$ of the $\langle 4\times 4\times 4 : 48\rangle$ schemes we measured.

DPS's programs are not the cheapest for their matrices. Section 8.4 computes `4x4x4_48_accurate_262`'s matrices in 260 operations. That changes no growth factor and no measurable time, and the measured error stays within about 2% at one to three levels, so it is a lower operation count and not a more accurate algorithm.

2 Background

2.1 Schemes, growth factors and Q_0

For an $m \times k$ matrix A, $\mathrm{Vect}(A)$ is its row-major vectorization. A bilinear algorithm with r multiplications for $\langle m \times k \times n\rangle$ is a triple of matrices $L \in \mathbb{Q}^{r\times mk}$, $R \in \mathbb{Q}^{r\times kn}$, $P \in \mathbb{Q}^{mn\times r}$ with

$$\mathrm{Vect}(AB) = P\big((L\,\mathrm{Vect}\,A) \odot (R\,\mathrm{Vect}\,B)\big) \tag{1}$$

for all A and B, where $\odot$ is the entrywise product. Here $m = k = n = 4$ and $r = 48$. With L_i the rows of L, R_i the rows of R, $p_{j,i}$ the entries of P and $q^\star$ the dual exponent of q, DPS's (p, q)-growth factor [12, Def. 11] is

$$\gamma_{p,q} = \Big\| \Big(\sum_{i=1}^{r} \|L_i\|_{q^\star} \|R_i\|_{q^\star} \, |p_{j,i}| \Big)_j \Big\|_p , \tag{2}$$

and the forward error of ℓ recursive levels is at most $f_{p,q}\|A\|_q\|B\|_q\,\varepsilon + O(\varepsilon^2)$ with $f_{p,q} = O(\gamma_{p,q}^\ell) = O(n^{\log_4 \gamma_{p,q}})$ [12, Th. 18]. The classical algorithm has $\gamma_{\infty,\infty} = \gamma_{\infty,2} = 4$ and $\gamma_{2,\infty} = \gamma_{2,2} = 16$, exponents 1 and 2. DPS also use the smooth bound $\gamma_2 = \sum_i \|L_i\|_2\|R_i\|_2\|P_{*,i}\|_2 \geq \gamma_{2,2}$ [11, eq. (5)]. The constant of their bounds also carries a combinatorial factor, after Ballard et al. [2, Def. 1] ([9, Th. 12], [12, Th. 18]):

$$Q_0 = \max_j \Big(\|P_j\|_0 + \max_{i:\, p_{j,i}\neq 0} \big(\|L_i\|_0 + \|R_i\|_0 \big) \Big), \tag{3}$$

where $\|\cdot\|_0$ counts nonzeros. For the worst output it counts the nonzeros of the worst product's two linear forms and the products summed into the output, a crude count of the roundings a level exposes the output to when the scheme is computed as direct sums: $Q_0 - 3$ additions and one multiplication. The bounds multiply it by $\gamma/(\gamma - k)$, which adds up the levels. PLinOpt's

`growthfactor` tool [14] (commit `cf85c3f`, May 2026, unchanged at `7c20065`) prints under the name Q_0 a different quantity, the term count

$$T_0 = \max_j \Big(\max_{i:\, p_{j,i} \neq 0} \|L_i\|_0 \, \|R_i\|_0 + \|P_j\|_0 \Big), \tag{4}$$

the terms $a_k b_l$ of the worst product feeding the output plus the products summed into it: for Winograd's $\langle 2 \times 2 \times 2 : 7 \rangle$ algorithm it prints 13 where [9] tabulates $Q_0 = 10$. Table 1 reports Q_0 as in (3); Section 7 uses T_0 once.

2.2 Isotropies and symmetries

For $(U, V, W) \in GL_4^3$, applying an algorithm to $U^{-1}AV$ and $V^{-1}BW$ returns $U^{-1}ABW$, so

$$(L, R, P) \;\mapsto\; \big(L\,(U^{-1} \otimes V^{\mathsf{T}}),\; R\,(V^{-1} \otimes W^{\mathsf{T}}),\; (U \otimes W^{-\mathsf{T}})\,P \big) \tag{5}$$

is again an algorithm, since $\mathrm{Vect}(XAY) = (X \otimes Y^{\mathsf{T}})\,\mathrm{Vect}\,A$ for row-major vectorization. These are de Groote's isotropies [7]; PLinOpt's orbit tool uses the same group. With $A_i = \mathrm{Mat}(L_i)$, $B_i = \mathrm{Mat}(R_i)$ and Z_i the 4×4 matrix of column i of P, (5) reads

$$A_i \mapsto U^{-\mathsf{T}} A_i V^{\mathsf{T}}, \qquad B_i \mapsto V^{-\mathsf{T}} B_i W^{\mathsf{T}}, \qquad Z_i \mapsto U Z_i W^{-1}. \tag{6}$$

Two further symmetries come from the trace form

$$\textstyle\sum_i \mathrm{tr}(X_i A)\,\mathrm{tr}(Y_i B)\,\mathrm{tr}(Z_i G) = \mathrm{tr}(ABG), \qquad X_i = A_i^{\mathsf{T}},\; Y_i = B_i^{\mathsf{T}} : \tag{7}$$

- the cyclic rotation $(X, Y, Z) \mapsto (Y, Z, X)$;
- the input swap $(X, Y, Z) \mapsto (Y^{\mathsf{T}}, X^{\mathsf{T}}, Z^{\mathsf{T}})$, from $(AB)^{\mathsf{T}} = B^{\mathsf{T}} A^{\mathsf{T}}$.

Every $\gamma_{p,q}$ treats L and R alike, through the norms of their rows, and P differently, through its entries and the norm over outputs. So the input swap leaves the growth factors unchanged, but the rotation does not: the three *role assignments*, one for each factor set that computes the outputs, are three different algorithms for accuracy purposes. γ_2 is symmetric in the three sets, and the rotation leaves it unchanged.

2.3 Invariances

Three facts organize the search.

- Scaling one product, $(L_i, R_i, P_{*,i}) \mapsto (\alpha L_i, \beta R_i, P_{*,i}/(\alpha\beta))$, changes no $\gamma_{p,q}$, no γ_2 and not Q_0. So none depends on the scale of U, V or W.
- For orthogonal U, V, W the Kronecker factors in (5) are orthogonal, so the 2-norms of the rows of L and R and of the columns of P are unchanged, and so is γ_2. Then $\gamma_{\infty,2} = \max_{p,q} \sum_i w_i |(U Z_i W^{\mathsf{T}})_{pq}|$ with fixed weights $w_i = \|L_i\|_2 \|R_i\|_2$, and V drops out.
- Multiplying U, V or W on the left by a signed permutation permutes rows and columns of the A_i, B_i, Z_i with signs. That changes no $\gamma_{p,q}$ and not Q_0.

We call the schemes of the orbit, in each role assignment, its *points*. Two points are *equivalent* when scalings of U, V, W, left multiplication of them by signed permutations, a relabelling and rescaling of the products, and the input swap take one to the other. Equivalent points have the same growth factors and the same Q_0, and when the rescalings are by powers of two, a program for one is, after the relabelling, a program for the other with the same roundings in its passes. So equivalent points are one algorithm written in different ways. We name a point after the published scheme it is equivalent to, if any.

Table 1: Growth factors, γ_2, Q_0 as in (3) and the operations of the published straight-line programs. In parentheses: the error-bound exponent $\log_k \gamma$ ($k = 2$ for $\langle 2\times 2\times 2 : 7\rangle$, $k = 4$ for $\langle 4\times 4\times 4 : 48\rangle$), the power of n with which the error bound grows. Classical multiplication has exponent 1 in the first two columns and 2 in the next two, whose output norm is the 2-norm over all n^2 entries rather than the largest entry. The first five rows are the algorithms of [11, Table 2] (values for `DPS-accurate`, whose coefficients involve $\sqrt{3}$, from that table). The sixth is `4x4x4_48_accurate_262`, in bold where it is the smallest among the $\langle 4\times 4\times 4 : 48\rangle$ algorithms with dyadic coefficients. The last two are points of the same orbit from our search and our earlier work [24]. Names are PLinOpt's without the prefixes `2x2x2_7_` and `4x4x4_48_`. Everything but the operations is a function of the matrices alone, so `accurate_260` (Section 8.4) shares the row of `accurate_262`, with 260 operations.

algorithm	$\gamma_{\infty,\infty}$	$\gamma_{\infty,2}$	$\gamma_{2,\infty}$	$\gamma_{2,2}$	γ_2	Q_0	ops
$\langle 2\times 2\times 2 : 7\rangle$							
`Winograd`	18 (4.170)	8 (3.000)	31.24 (4.965)	14 (3.807)	17.85	10	15
`Strassen`	12 (3.585)	6.83 (2.772)	17.89 (4.161)	10.45 (3.386)	14.83	8	18
`DPS-accurate` [9]	17.48 (4.128)	5.97 (2.577)	27.71 (4.793)	10.01 (3.323)	–	15	–
$\langle 4\times 4\times 4 : 48\rangle$							
`rational` [10]	288 (4.085)	38.16 (2.627)	1032.0 (5.006)	139.9 (3.564)	210.5	48	315
`accurate` [11]	148 (3.605)	25.46 (2.335)	514.9 (4.504)	91.21 (3.256)	154.5	46	284
`accurate_262`	**132** (3.522)	**22.63** (2.250)	**496.5** (4.478)	**90.51** (3.250)	154.5	**34**	262
$\gamma_{\infty,2}$ min. (other roles), $\mathbb{Q}(\sqrt{2})$	176.6 (3.732)	21.41 (2.210)	583.8 (4.595)	85.63 (3.210)	157.3	42	–
`201` [24]	399 (4.320)	75.85 (3.123)	1072.5 (5.033)	225.9 (3.910)	369.7	38	201

3 The 262-operation variant

`4x4x4_48_accurate_262` was added to PLinOpt on 13 October 2026. Its straight-line programs take 64 additions and 8 scalings for L, 67 and 11 for R, and 112 additions and no scalings for P, against `4x4x4_48_accurate`'s 80+4, 68+8 and 108+16. Up to symmetry it is `4x4x4_48_accurate` moved by an orthogonal isotropy with entries $0, \pm 1/\sqrt{2}$. Take `4x4x4_48_accurate`'s (L, R, P) in its published role and apply (5) with $(U, V, W) = (Q_U, Q_V, Q_W)/\sqrt{2}$, where

$$Q_U = \begin{pmatrix} 0 & -1 & 0 & -1 \\ 1 & 0 & -1 & 0 \\ 0 & -1 & 0 & 1 \\ -1 & 0 & -1 & 0 \end{pmatrix}, \quad Q_V = \begin{pmatrix} -1 & -1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}, \quad Q_W = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & -1 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}. \tag{8}$$

Each Kronecker factor in (5) is then a $\{0, \pm 1\}$ matrix divided by 2, so the result has coefficients in $\mathbb{Z}[\frac{1}{2}]$ with denominators at most 4, like `4x4x4_48_accurate`. Mapping `4x4x4_48_accurate_262` through the input swap and signed permutations of U, V, W, and relabelling and rescaling the products, gives exactly this scheme.

`4x4x4_48_accurate_262`'s coefficients other than ± 1 are $\pm 1/2$ (224 in L and 80 in R), $\pm 1/4$ (248 in R) and $\pm 3/4$ (8 in R); every entry of P is ± 1. The $\pm 3/4$ belong to the scheme, not to this way of writing it. In eight products one entry of the B side is three times the others; the B side of product 32 in PLinOpt's file, for instance, is

$$\tfrac{1}{4}\,(b_{11} - 3b_{13} - b_{21} - b_{23} + b_{31} + b_{33} + b_{41} + b_{43}). \tag{9}$$

Rescaling a product scales its whole row, so every scheme equivalent to `4x4x4_48_accurate_262` has this ratio. DPS's programs never multiply by 3 or 3/4: their only scalings are 19 halvings, so every coefficient is a sum of terms halved different numbers of times.

Proposition 3.1. *`4x4x4_48_accurate_262` has* $\gamma_{\infty,\infty} = 132$, $\gamma_{\infty,2} = 16\sqrt{2}$, $\gamma_{2,\infty} = 16\sqrt{963} \approx 496.52$, $\gamma_{2,2} = 64\sqrt{2}$, $\gamma_2 = 64(1+\sqrt{2})$ *and* $Q_0 = 34$. *`4x4x4_48_accurate` has* 148, $18\sqrt{2} \approx 25.456$, $8\sqrt{4142} \approx 514.87$, $8\sqrt{130} \approx 91.214$, $64(1+\sqrt{2})$ *and* 46.

The values are exact. Every weight $\|L_i\|_2\|R_i\|_2$ and every column norm of P is a rational or a rational multiple of $\sqrt{2}$, so the sums lie in $\mathbb{Q}(\sqrt{2})$, and we computed them, and the rational squares of $\gamma_{2,\infty}$ and $\gamma_{2,2}$, in exact arithmetic. PLinOpt's `growthfactor`, working in floating point, agrees to every printed digit. Table 1 sets them beside the other algorithms of [11, Table 2] and two points from our search. The error-bound exponents $\log_4 \gamma_{\infty,2}$ and $\log_4 \gamma_{2,2}$ are exactly 9/4 and 13/4. Both algorithms' published programs pass PLinOpt's `SLPchecker` against their matrices, and both pass our exact check against the matrix multiplication tensor. The proposition depends on the matrices only, so it also holds for the 260-operation programs of Section 8.4.

4 How we found it

4.1 Role assignments

The three role assignments of `4x4x4_48_accurate` have $\gamma_{\infty,2} = 25.456$, 32.971 and 28.142 ($\gamma_{\infty,\infty} =$ 148, 212, 180), so the published one is already the best. For `4x4x4_48_rational` they give 38.162, 39.282 and 33.540, and the published one is not the best. Every search below runs from all three.

4.2 Continuous search over the orbit

We minimize $\gamma_{\infty,2}$ over $(U,V,W) \in GL_4^3$, that is, over all 48 parameters, by brute force. This is not how such a search should be done, and we describe the search as we ran it only so that its results can be checked. The orbit has structure that the objective respects, and DPS's own searches use it [9, Section 4]: orthogonal isotropies leave γ_2 unchanged (their Lemma 13), so the search for the growth factor is restricted to the factorization of GL_4 by the orthogonal matrices, isotropies of the form $H_\rho P_\xi$ with H_ρ a positive diagonal matrix and P_ξ unipotent (their Lemma 14; Lemma 27 of [12]), and on that quotient, Sedoglavic tells us, a convexity result makes the minimum unique [22]. The norms that orthogonal isotropies do change, $\gamma_{\infty,2}$ among them, are then searched over the orthogonal matrices alone, with $SO(4)$ parametrized by Van Elfrinkhof's formula [6], every rotation the product of a left- and a right-isoclinic rotation with three parameters each: 18 parameters for (U,V,W) instead of 48. (He also points to earlier work by Turner on this parametrization; reference to be supplied.) Our brute-force search found the same minima (its optima have two orthogonal factors and one of the form orthogonal times diagonal, Section 4), at a cost that the restricted search would have divided by a large factor, and anyone repeating a search of this kind should start from the restricted form.

The max over the 16 outputs is smoothed by a log-sum-exp with sharpness β and $|x|$ by $\sqrt{x^2+\epsilon^2}$, the smoothed objective is minimized by BFGS with a backtracking line search, and (β,ϵ) runs through a continuation from $(0.5, 10^{-1})$ to $(32768, 10^{-6})$. After each stage each of U, V, W is rescaled to determinant ± 1, which changes nothing (Section 2.3). The gradient is exact: with $M_i = U^{-\mathsf{T}} A_i V^{\mathsf{T}}$, $N_i = V^{-\mathsf{T}} B_i W^{\mathsf{T}}$, $Z_i' = U Z_i W^{-1}$, $a_i = \|M_i\|_F$, $b_i = \|N_i\|_F$ and π the softmax weights of the outputs, the adjoints are

$$\frac{\partial f}{\partial Z_i'} = \pi \circ a_i b_i \, s'(Z_i'), \qquad \frac{\partial f}{\partial M_i} = \frac{b_i S_i}{a_i} M_i, \qquad \frac{\partial f}{\partial N_i} = \frac{a_i S_i}{b_i} N_i, \tag{10}$$

with $S_i = \sum_{pq} \pi_{pq} s(Z'_{i,pq})$. They are carried through the products and through each inverse by $\partial X = -X \, \partial M \, X$. The implementation (`gamma48`, in Rust) checks this gradient against central

differences to 10^{-10} relative. It runs on all cores, and one start takes about 0.5 s on one core. Starts are the identity, random near-identity points, and random rotations times random perturbations of the identity.

The orthogonal part of the group is where our search differs from DPS's. Their objective is γ_2, which orthogonal U, V, W leave unchanged (Section 2.3), so their minimization discards that freedom by an Iwasawa-type reduction and keeps a scaling and a shear per factor [9, 12]; orthogonal matrices enter their work only afterwards, to sparsify. Neither of our measures is invariant: $\gamma_{\infty,2}$ and the amplification of Section 8.2 depend on the entries of UZ_iW^{T}, not on their 2-norms, so the orthogonal sub-orbit is not a degeneracy to be quotiented out but the place where the minimum can move. To search it without constraints we write each rotation as a Cayley transform, $Q = (I - S)(I + S)^{-1}$ with S skew-symmetric, Cayley's construction of 1846 [5]: Q is orthogonal for every S, every rotation without a -1 eigenvalue arises from exactly one S, and the six free entries of S replace six constrained ones of Q, so the optimizer over the twelve entries of two skew matrices stays on the sub-orbit at every step. The transform is classical; its use here, as the coordinates of an accuracy search that DPS's measure cannot see, is what makes the sub-orbit minimum of the next paragraph findable.

Within the orthogonal sub-orbit (γ_2 fixed at its minimum 154.51; 12 parameters, two Cayley-parametrized rotations), the minimum is $16\sqrt{2} \approx 22.627$ for the published role and $12 + 8\sqrt{2} \approx 23.314$ for the other two. At the minimum U and W have entries $0, \pm 1/\sqrt{2}$ to machine precision, so the optimum is an exact point. Rescaling the products makes its coefficients dyadic (Section 4.3), and it is equivalent to `4x4x4_48_accurate_262`. Over the whole orbit, 30,000 starts from `4x4x4_48_accurate` (10,000 per role) and 7,200 each from `4x4x4_48_rational` and from `4x4x4_48_201` give the same local optima, a direct sign that the three lie in one orbit. The role decides what the search finds:

- In the published role the lowest optimum is $16\sqrt{2}$, reached 248 times in `4x4x4_48_accurate`'s 10,000 starts, and nothing lower appears. The lowest point the search finds in this role is equivalent to `4x4x4_48_accurate_262`, and exact as found: we did not reach it by snapping a lower point to a dyadic grid.

- In the other two roles the lowest optimum, 21.4073, is found thousands of times, and nothing lower appears; the next is 21.944. Its points have two orthogonal factors and one with singular values $(1, 1, s, s)$, $s = 2(\sqrt{2} - 1)$, so their coefficients lie in $\mathbb{Q}(\sqrt{2})$. They cannot be made dyadic: the scaling enters R as s and P as $1/s$, and no dyadic value other than a power of two has a dyadic inverse. Snapping them to the dyadic orthogonal points of their roles would not help either: those have $\gamma_{\infty,2} \geq 12 + 8\sqrt{2}$ (Section 4.3), more than `4x4x4_48_accurate_262`'s $16\sqrt{2}$.

The point we measured (Section 5) is exact in that field. Snapping each coefficient of its direct-sum programs to the nearest $a + b\sqrt{2}$ with $a, b \in \frac{1}{4}\mathbb{Z}$ moves none by more than $1.2 \cdot 10^{-8}$, and the result satisfies the Brent equations exactly in $\mathbb{Q}(\sqrt{2})$. Its L has 304 nonzero coefficients, all $\pm\frac{1}{2}$; its R has 384, half $\pm\frac{1}{2}$ and half $\pm(\sqrt{2} - 1)$; its P has 240: $\pm\frac{1}{2}$, ± 1, ± 2, $\pm\frac{1}{4}(1 + \sqrt{2})$, $\pm\frac{1}{2}(1 + \sqrt{2})$ and $\pm(1 + \sqrt{2})$ (80, 40, 24, 16, 56 and 24 of them). Every $\|R_i\|_2^2$ is $13 - 8\sqrt{2}$ and every $\|L_i\|_2^2$ is rational, so each output's sum $\sum_i |p_{j,i}| \, \|L_i\|_2 \|R_i\|_2$ collects to $(8 + 6\sqrt{2})\sqrt{13 - 8\sqrt{2}}$, the same for all sixteen outputs:

$$\gamma_{\infty,2} = (8 + 6\sqrt{2})\sqrt{13 - 8\sqrt{2}} = 2\sqrt{58 + 40\sqrt{2}} \approx 21.40734, \tag{11}$$

an algebraic number of degree four, with exponent $\log_4 \gamma_{\infty,2} \approx 2.2100$; and $\gamma_{\infty,\infty} = 120 + 40\sqrt{2} \approx 176.57$. The exact scheme, the programs we measured and the search's (U, V, W) are in the repository (Section 9).

4.3 The dyadic orthogonal sub-orbit, exhaustively

A scheme is exact in binary floating point only if its coefficients are dyadic, and a scaling is exact only if it is by a power of two: multiplying by another dyadic constant, such as 3/4, rounds. `4x4x4_48_accurate_262`'s R has eight coefficients $\pm 3/4$ and `4x4x4_48_accurate`'s has sixteen, but DPS's programs for them never multiply by 3/4. They build it from halvings and additions, so all their rounding is in additions. Isotropies with dyadic entries and dyadic inverses keep a scheme dyadic, and up to scale they are dense in $\mathrm{GL}_4(\mathbb{R})$, since they include the elementary matrices $I + tE_{ij}$ for every dyadic t. So dyadic points come arbitrarily close to the $\mathbb{Q}(\sqrt{2})$ minimum, but only with coefficients of ever more bits, and multiplying by those rounds just as multiplying by s does. We look instead for points with small dyadic coefficients. Let G be the set of orthogonal 4×4 matrices whose entries lie in $\mathbb{Z}[\frac{1}{2}]$ or in $\mathbb{Z}[\frac{1}{2}]/\sqrt{2}$. It has 2304 elements:

- the 384 signed permutations and the 768 matrices with entries $\pm 1/2$, which together form the Weyl group $W(F_4)$;
- the 1152 matrices with entries $0, \pm 1/\sqrt{2}$.

It is closed under products and inverses. For $(U, V, W) \in G^3$ the three Kronecker factors of (5) carry the factors $\sqrt{2}^{-(c_U+c_V)}$, $\sqrt{2}^{-(c_V+c_W)}$ and $\sqrt{2}^{-(c_U+c_W)}$, with $c = 1$ for the $1/\sqrt{2}$ matrices and 0 otherwise. Their product is a power of two, so scaling each product makes every coefficient dyadic. By Section 2.3 the growth factors and Q_0 depend only on the classes of U, V, W under left multiplication by signed permutations. There are six classes of 384, so the sub-orbit of each role has 216 distinct points, 648 in all, and we enumerate them.

Theorem 4.1. *Among the* 648 *points of the dyadic orthogonal sub-orbit of* `4x4x4_48_accurate`*'s scheme, the minimum of* $\gamma_{\infty,2}$ *is* $16\sqrt{2}$*, attained at* 42 *points, all in the published role; in each of the other two roles it is* $12 + 8\sqrt{2}$*, attained at* 36 *points. The minimum of* $\gamma_{\infty,\infty}$ *is* 132*, attained at exactly eight points, all in the published role, and these eight also attain* $\gamma_{\infty,2} = 16\sqrt{2}$*. Each of the eight is equivalent to* `4x4x4_48_accurate_262`*.*

The proof is the enumeration together with an explicit equivalence for each of the eight points (`equiv.py`). `4x4x4_48_accurate`'s own point has the fourth smallest $\gamma_{\infty,2}$ in the sub-orbit (25.456, after $16\sqrt{2}$, $12 + 8\sqrt{2}$ and 24). So within the dyadic orthogonal moves of `4x4x4_48_accurate`, the points that are best for both growth factors are all equivalent to `4x4x4_48_accurate_262`.

4.4 Other classes

Rational rank-48 schemes are not all in one orbit. Li, Wang and Hu put DPS's scheme on a curve $s(t)$ whose members are pairwise inequivalent for $t \geq 1/4$ [18], with the invariant $\varphi = 64 + 4/t^2 + 1024t^2$; the DPS class is $t = 1/4$, the minimum of φ. For $t = 2^k$ the coefficients are dyadic. Agarwal replaced twelve products of $s(2)$ to get a scheme off the curve [1]. We rebuilt both exactly from the papers' statements (checked on all 4096 Brent equations) and ran the continuous search from each, 2400 starts per role (Table 2). The smallest $\gamma_{\infty,2}$ grows steadily with t. No other class has any point, dyadic or not, as low as `4x4x4_48_accurate_262`'s 22.63; the closest, $s(1/2)$, bottoms out at 23.14.

Table 2: Smallest $\gamma_{\infty,2}$ found over the whole orbit of each rational class, 2400 starts per role, and the Li–Wang–Hu invariant φ. The role numbering follows each scheme's own coordinates.

class	φ	role 0	role 1	role 2	minimum
$s(1/4)$, the DPS class	192	21.407	22.627	21.407	21.407
$s(1/2)$	336	23.144	23.654	23.144	23.144
$s(1)$	1092	25.167	26.833	25.167	25.167
$s(2)$	4161	27.852	32.485	27.852	27.852
$s(4)$	16448.25	32.677	40.319	32.677	32.677
$s(8)$	65600.06	40.134	51.351	40.134	40.134
Agarwal's scheme	–	39.619	42.924	39.619	39.619

5 Accuracy

5.1 Implementation

The measurements use our fast matrix multiplication library for Apple silicon [25]. One level of a $\langle 4 \times 4 \times 4 : 48 \rangle$ scheme runs as three fused passes, each one streaming loop over the elements of the blocks:

- the left pass computes all 48 combinations $L \operatorname{Vect} A$ of the 16 blocks of A;
- the right pass does the same for B;
- the output pass combines the 48 block products into the 16 blocks of C.

The block products go to Accelerate's GEMM (`cblas_sgemm`, `cblas_dgemm`). The passes are generated from the published straight-line programs, operation by operation in the programs' order, so each program's rounding is the one measured. Deeper levels recurse on the block products. One, two and three levels at $n = 4096$ leave GEMMs of size 1024, 256 and 64, and at $n = 8192$ sizes 2048, 512 and 128. For comparison we include `2x2x2_7_Winograd` (Strassen–Winograd, 15 additions) at one, two and four levels. Two of its levels reduce the size as much as one $\langle 4 \times 4 \times 4 \rangle$ level.

5.2 Protocol

- Inputs are uniform on $[-1, 1)$ or standard normal, from a xorshift generator. Each setting uses five seeds, three for double precision at $n = 8192$.
- Single precision is measured against the double-precision product of the same inputs (Accelerate's `dgemm`). Its 29 extra bits make it exact for this purpose.
- Double precision is measured against a compensated product: every entry is an Ogita–Rump–Oishi Dot2 dot product [20], with each product split exactly by a fused multiply-add and the sum accumulated by error-free TwoSum steps, then rounded to double. Its own error is about the unit roundoff, two orders of magnitude below the errors measured.
- The error is $\max_{ij} |\hat{C}_{ij} - C_{ij}| / \max_{ij} |C_{ij}|$. We report it as a multiple of the classical GEMM's own error on the same inputs.

The programs are:

- `4x4x4_48_accurate` with DPS's 284-operation programs;

- `4x4x4_48_accurate_262` with DPS's 262-operation programs;
- `4x4x4_48_accurate_262` with our programs. Our scheme is equivalent to `4x4x4_48_accurate_262`: the input swap, signed permutations and a rescaling of the products take one to the other. Ours has its constants $\pm 1/2$ and $\pm 1/4$ moved into P: no entry of its P is ± 1, and its L has ± 2, ± 3, $\pm 3/2$ and $\pm 1/2$. Its programs come from one default run of PLinOpt's `optimizer`, L and R directly and P through PLinOpt's transposition pipeline (`matrix-transpose`, `optimizer -K`, `transpozer`): 445 operations. The optimizer is randomized, and a rerun gives 437. The same run on DPS's scheme gives 383, so how the scheme is written accounts for about 60 of the 180 operations between our programs and DPS's. A sweep of the optimizer's options on both schemes (Section 7) reaches 348 and 409;
- `4x4x4_48_rational`'s 315-operation programs and the 201-operation programs of [24];
- the orbit's $\gamma_{\infty,2}$ minimum, in another role, written as direct sums with its coefficients rounded to the working precision;
- `4x4x4_48_accurate` written as direct sums, a control for program style; unlike DPS's programs, these multiply by 3/4 directly;
- two dyadic points of `4x4x4_48_accurate`'s orbit, written as direct sums, chosen from the enumeration of Section 4.3 to separate the two growth factors. Probe A has `4x4x4_48_accurate_262`'s $\gamma_{\infty,2} = 16\sqrt{2}$ but $\gamma_{\infty,\infty} = 256$ (role 0, classes $5, 2, 3$); probe B has $\gamma_{\infty,2} = 28.97$, larger than `4x4x4_48_accurate`'s, but $\gamma_{\infty,\infty} = 144$, smaller (role 2, classes $4, 4, 2$). Their Q_0 are 40 and 46, and their coefficients are ± 1, $\pm 1/2$ and $\pm 1/4$, so their direct sums scale only by powers of two.

5.3 Results

Table 3 gives each scheme's mean error as a multiple of the classical GEMM's. Table 4 gives `4x4x4_48_accurate_262`'s error divided by `4x4x4_48_accurate`'s on the same inputs, and Figure 1 plots the means.

- `4x4x4_48_accurate_262` with DPS's programs is more accurate than `4x4x4_48_accurate` at every depth, in both precisions and for both distributions. Its error is 0.73–0.83 of `4x4x4_48_accurate`'s averaged over the seeds of each setting, 0.75–0.77 averaged over all instances, and 0.62–0.90 for single instances. Of the programs of Section 5.2 it has the lowest error at two and three levels. At one level `4x4x4_48_accurate_262` with our programs (0.71 in single precision) and probe A (0.71 in double) are slightly lower than its 0.73 and 0.72.
- At one and two levels, where the library's plans run, it is more accurate than the classical GEMM itself (0.72–0.81 of its error), while `4x4x4_48_accurate` is about as accurate as the classical GEMM (0.94–1.05). The block products are dot products a quarter or a sixteenth as long, which saves more than the passes add.
- Single and double precision agree within a few percent, and so do the two sizes and the two distributions.
- Over these depths the advantage is nearly constant: from two levels to three both errors grow by a factor of 1.46–1.47 in single precision (1.52 for `4x4x4_48_accurate` and 1.43 for the variant in double). With large leaves the leaf GEMMs' error dominates, and the difference

Table 3: Error against the exact reference as a multiple of the classical GEMM's own error, mean over all instances: single precision 20 instances ($n = 4096$ and 8192, uniform and normal, five seeds each); double precision 16 ($n = 4096$, five seeds per distribution; $n = 8192$, three), except `accurate`'s direct sums, the probes and `201`, which have the 10 at $n = 4096$ only. Winograd's levels are binary, so its columns are two and four levels; "–": not measured.

	single precision			double precision		
levels	1	2	3	1	2	3
`accurate`, DPS's programs (284)	0.94	1.05	1.54	0.94	1.03	1.57
`accurate_262`, DPS's programs (262)	0.73	0.79	1.17	0.72	0.81	1.16
`accurate_260`, our programs (260)	0.74	0.82	1.18	0.72	0.82	1.16
`accurate_262`, DPS's programs, f64 passes	0.72	0.80	1.14	–	–	–
`accurate_262`, model-searched programs (539)	0.73	0.80	1.18	0.72	0.81	1.18
`accurate_262`, our programs (445)	0.71	0.83	1.33	0.73	0.83	1.32
`accurate`, direct sums	0.96	1.07	1.65	0.93	1.08	1.70
probe A, direct sums	0.74	0.82	1.32	0.71	0.85	1.36
probe B, direct sums	0.78	0.82	1.26	0.74	0.86	1.36
orbit's $\gamma_{\infty,2}$ minimum (other roles), $\mathbb{Q}(\sqrt{2})$	0.86	1.00	1.61	0.88	1.01	1.57
`rational`, DPS's programs (315)	1.06	1.60	3.27	1.03	1.56	3.23
`201`	2.78	8.72	36.84	2.59	8.74	36.77
`2x2x2_7_Winograd`, 2 and 4 binary levels	–	3.11	10.25	–	3.00	9.93

Table 4: Error of `accurate_262` (DPS's programs) divided by `accurate`'s on the same inputs: mean over the seeds, with the range in brackets.

precision	setting	1 level	2 levels	3 levels
single	$n = 4096$, uniform	0.82 [0.75, 0.90]	0.74 [0.67, 0.81]	0.78 [0.76, 0.82]
single	$n = 4096$, normal	0.76 [0.64, 0.88]	0.81 [0.77, 0.84]	0.75 [0.66, 0.80]
single	$n = 8192$, uniform	0.77 [0.69, 0.85]	0.74 [0.67, 0.79]	0.76 [0.70, 0.78]
single	$n = 8192$, normal	0.77 [0.73, 0.81]	0.73 [0.62, 0.77]	0.75 [0.68, 0.85]
double	$n = 4096$, uniform	0.79 [0.73, 0.86]	0.78 [0.72, 0.83]	0.73 [0.70, 0.82]
double	$n = 4096$, normal	0.77 [0.71, 0.83]	0.78 [0.75, 0.85]	0.76 [0.67, 0.89]
double	$n = 8192$, uniform	0.73 [0.64, 0.83]	0.83 [0.80, 0.88]	0.74 [0.74, 0.74]
double	$n = 8192$, normal	0.75 [0.68, 0.81]	0.78 [0.75, 0.81]	0.76 [0.70, 0.83]

comes from the passes' constant (Q_0), not yet from the growth factor. Section 5.4 recurses down to scalars to see the growth.

5.4 Deep recursion

To see the growth factors at work we recursed down to scalars: $n = 1024$ with one to five levels (leaves 256, 64, 16, 4 and 1), as in DPS's experiments [11, Fig. 1], ten instances per precision. Table 7 gives the results and Figure 2 plots them.

- Of the programs of Section 5.2, `4x4x4_48_accurate_262` with DPS's programs is again the most accurate from two levels on, and at one level in double precision; at one level in single precision probe A and our programs for `4x4x4_48_accurate_262` have 0.78 against its 0.79. Its advantage over `4x4x4_48_accurate` grows with depth: its error is 0.80–0.83 of `4x4x4_48_accurate`'s at one level and 0.72 at five (single instances 0.67–0.80), in both precisions.

Table 5: The factors beside the measured single-precision error at three levels (a multiple of the classical GEMM's error, mean over all instances) and its growth from two levels to three. Sorted by the measured error.

scheme	$\gamma_{\infty,2}$	$\gamma_{\infty,\infty}$	Q_0	program	3 levels	growth
`accurate_262`	22.63	132	34	DPS, 262	1.17	1.47
probe B	28.97	144	46	direct sums	1.26	1.53
probe A	22.63	256	40	direct sums	1.32	1.62
`accurate_262`	22.63	132	34	ours, 445	1.33	1.60
`accurate`	25.46	148	46	DPS, 284	1.54	1.46
orbit's $\gamma_{\infty,2}$ min. (other roles)	21.41	176.6	42	direct sums	1.61	1.61
`accurate`	25.46	148	46	direct sums	1.65	1.55
`rational`	38.16	288	48	DPS, 315	3.27	2.05
`201`	75.85	399	38	ours, 201	36.84	4.22

Table 6: Time of DPS's programs in our library: the best of three alternating rounds, each the median of five runs.

precision	plan, n	`accurate`	`accurate_262`	ratio
single	`D48/G`, 4096	0.044 s	0.044 s	1.000
single	`D48/G6`, 4096	0.036 s	0.036 s	1.000
single	`D48+4/D48/G6`, 8192	0.236 s	0.237 s	1.004
single	`D48/G`, 8192	0.272 s	0.271 s	0.996
double	`D48/G`, 4096	0.139 s	0.140 s	1.007
double	`D48/G6`, 4096	0.177 s	0.173 s	0.977
double	`D48+4/D48/G6`, 8192	0.800 s	0.803 s	1.004
double	`D48/G`, 8192	1.054 s	1.061 s	1.007

- From four levels to five, the errors grow by 2.60–2.61 for `4x4x4_48_accurate_262` and 2.75–2.79 for `4x4x4_48_accurate`, about 6% slower per level. That is the direction $\gamma_{\infty,2}$ predicts, but less than its ratio $16\sqrt{2}/18\sqrt{2} = 8/9$ would give.
- Programs matter more at depth. `4x4x4_48_accurate_262` with our 445-operation programs and the two probes beat `4x4x4_48_accurate` in the main campaign, where the leaves are larger, but at $n = 1024$ they fall behind `4x4x4_48_accurate` with DPS's programs from three levels on: 2.12–2.29 against 2.00 at three levels and 13.96–15.45 against 13.29 at five, in single precision. Of the programs of Section 5.2, only `4x4x4_48_accurate_262` with DPS's programs beats `4x4x4_48_accurate` at every depth. Section 7 explains why.
- Among the direct-sum programs, the dyadic probes stay 24–29% below `4x4x4_48_accurate`'s at five levels and the orbit's $\gamma_{\infty,2}$ minimum only about 8%: the rounding of its constants compounds with depth.
- `4x4x4_48_rational` grows by about 3.6 per level, `4x4x4_48_201` by about 7.5, and `2x2x2_7_Winograd` by about 7 per two binary levels.

6 Speed

Table 6 times DPS's programs in our library. It covers single levels with one GEMM at a time (`D48/G`) and the plans the library picks at these sizes, which use six GEMM streams and pipelined

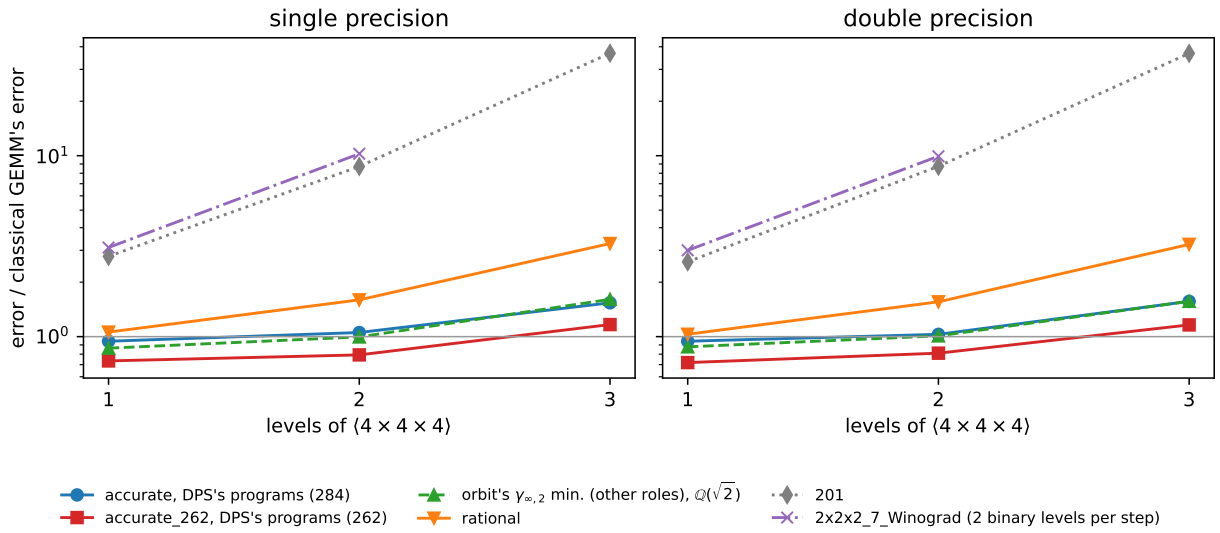


Figure 1: Mean error against the exact reference as a multiple of the classical GEMM's error, over all sizes, distributions and seeds of the main campaign ($n = 4096$ and 8192). The points for `2x2x2_7_Winograd` are two and four binary levels.

Table 7: Deep recursion at $n = 1024$: mean error against the exact reference as a multiple of the classical GEMM's error, over ten instances per precision (uniform and normal, five seeds each); five levels reach scalar leaves. In parentheses: operation counts of DPS's programs, or of ours where marked. Winograd: twice as many binary levels.

	single precision					double precision				
levels	1	2	3	4	5	1	2	3	4	5
`accurate` (284)	0.95	1.10	2.00	4.77	13.29	0.91	1.18	2.09	4.91	13.50
`accurate_262` (262)	0.79	0.97	1.67	3.75	9.79	0.76	0.98	1.63	3.68	9.58
`accurate_260`, ours (260)	0.80	0.96	1.67	3.61	9.20	0.77	0.96	1.62	3.54	9.19
`accurate_262`, f64 passes	0.77	0.93	1.42	2.70	6.25	–	–	–	–	–
`accurate_262`, searched (539)	0.78	0.98	1.68	3.54	9.25	0.77	0.92	1.62	3.50	8.91
`accurate_262`, ours (445)	0.78	1.10	2.29	5.71	15.45	0.78	1.09	2.20	5.48	15.62
`accurate`, direct	1.00	1.35	2.69	6.76	19.63	0.95	1.29	2.57	6.62	19.36
probe A	0.78	1.08	2.27	5.40	14.91	0.80	1.09	2.21	5.43	14.45
probe B	0.82	1.08	2.12	5.28	13.96	0.82	1.05	2.12	5.10	14.05
$\gamma_{\infty,2}$ min. (other roles), $\mathbb{Q}(\sqrt{2})$	0.89	1.22	2.40	6.42	18.06	0.92	1.20	2.36	5.89	17.57
`rational`	1.08	1.92	4.76	15.48	56.64	1.13	1.90	4.80	15.36	55.20
`201`	2.75	9.86	48.87	304	2242	2.89	9.60	47.50	301	2293
`Winograd`	2.88	10.46	46.34	254	1726	3.38	11.14	47.75	248	1753

passes (`D48/G6`, `D48+4/D48/G6`), in alternating rounds. Both take the same time to within the timer's resolution (0.977–1.007). The passes move the same data, and with 262 against 284 operations neither is bound by arithmetic. Our 445-operation programs for `4x4x4_48_accurate_262` also took `4x4x4_48_accurate`'s time. In an earlier run of three alternating rounds of five, their fastest run was 0.998–1.007 times `4x4x4_48_accurate`'s in seven of the eight configurations, and 1.041 in the eighth (double precision, `D48/G6`, $n = 4096$), where the medians of the rounds vary by up to 10%. Their passes are slower, 0.057 against 0.054–0.055 s at $n = 8192$ in double precision, but the passes take only 5–17% of the time. So the more accurate algorithm costs nothing at run

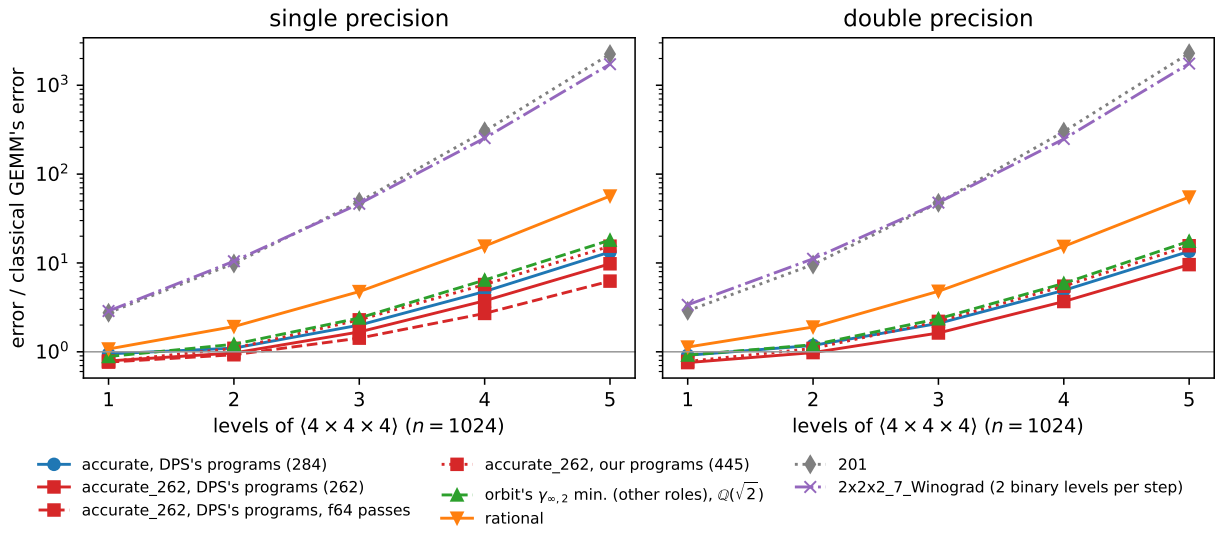


Figure 2: Deep recursion at $n = 1024$: mean error against the exact reference as a multiple of the classical GEMM's error, over ten instances per precision. The points for `2x2x2_7_Winograd` are two binary levels per step.

time, and it is the cheaper one in operation count.

7 Discussion

What predicts the measured error. Table 5 sets the factors beside the error measured at three levels in single precision, where the schemes differ most. The growth factors separate the distant schemes (`4x4x4_48_rational`, `4x4x4_48_201`, two levels of `2x2x2_7_Winograd`), but no single factor orders the close ones.

- Probe A has `4x4x4_48_accurate_262`'s $\gamma_{\infty,2}$ but a larger measured error.
- Probe B has a larger $\gamma_{\infty,2}$ than `4x4x4_48_accurate` but a smaller measured error.
- The orbit's $\gamma_{\infty,2}$ minimum, in another role, has the smallest $\gamma_{\infty,2}$ of all but about the same measured error as `4x4x4_48_accurate`.

Among the dyadic schemes, Q_0 does not order the measured errors either: 34, 46, 40 and 46 for `4x4x4_48_accurate_262`, probe B, probe A and `4x4x4_48_accurate`. The term count T_0 of (4) does, 80, 94, 144 and 180, on these four points. Of the dyadic schemes, `4x4x4_48_accurate_262` is best or tied for best in every factor, and with DPS's programs its measured error is the smallest.

Dyadic coefficients. The orbit's $\gamma_{\infty,2}$ minimum and probe A have nearly the same $\gamma_{\infty,2}$ (21.41 and 22.63) and Q_0 (42 and 40), and the former has the smaller $\gamma_{\infty,\infty}$ (176.6 against 256). Yet the former's measured error is about 20% larger at three levels: 1.61 against 1.32 times the classical error in single precision, and 1.57 against 1.36 in double. Its constants, such as $2(\sqrt{2}-1)$, are rounded to the working precision, so the scheme itself is inexact by a unit roundoff, and each multiplication by them rounds. Multiplications by $\pm 1/2^k$ are exact. The gap compounds with depth: at five levels the probes are 24–29% below `4x4x4_48_accurate`'s direct sums and the $\mathbb{Q}(\sqrt{2})$ point only 8%. A

Table 8: The first-order rounding model of one level of each program (Section 7) beside the measured errors. L+R and P: the RMS error those passes add, mean over the outputs; worst: all passes, worst output; all three relative to an output's RMS, in units of one rounding. Count and size: the roundings of the P pass that reach an output and their RMS size in output units (value times weight), means over the outputs; an output's P error is $\sqrt{\text{count}}$ times the size. Amp.: how much the worst output amplifies the leaf products' own errors, a property of the scheme. The last two rows are the best program sets of the optimizer sweep (Section 7), modelled but not measured. Measured: single-precision error as a multiple of the classical GEMM's at three levels (Table 3) and five (Table 7).

	model, one level						measured	
program	L+R	P	count	size	worst	amp.	3 levels	5 levels
`accurate_262`, DPS's programs (262)	5.74	3.09	15.0	0.80	7.22	2.83	1.17	9.79
`accurate_260`, our programs (260)	5.67	3.02	15.0	0.78	7.12	2.83	1.18	9.20
`accurate_262`, model-searched programs (539)	5.52	2.97	15.0	0.77	6.47	2.83	1.18	9.25
`accurate_262`, our programs (445)	5.61	8.22	35.6	1.39	12.51	2.83	1.33	15.45
probe B, direct sums	7.19	7.42	29.0	1.38	10.75	2.83	1.26	13.96
probe A, direct sums	10.12	5.67	15.0	1.46	11.77	2.83	1.32	14.91
`accurate`, DPS's programs (284)	5.57	3.35	18.0	0.79	7.12	3.16	1.54	13.29
`accurate`, direct sums	7.64	5.98	18.0	1.41	11.14	3.16	1.65	19.63
`rational`, DPS's programs (315)	7.75	7.11	47.8	1.04	11.73	4.00	3.27	56.64
`201`, our programs (201)	12.30	8.87	22.7	1.85	21.05	9.53	36.84	2242
`accurate_262`, DPS's scheme, sweep (348)	7.05	3.15	15.0	0.81	8.95	2.83	–	–
`accurate_262`, our scheme, sweep (409)	7.39	3.40	15.0	0.88	9.96	2.83	–	–

search for accuracy should therefore keep to small dyadic coefficients, and its programs to scalings by powers of two.

Programs. The same scheme has different measured errors with different programs. `4x4x4_48_accurate_262` with DPS's programs beats `4x4x4_48_accurate_262` with ours, and `4x4x4_48_accurate` with DPS's programs beats `4x4x4_48_accurate` as direct sums (1.17 against 1.33, and 1.54 against 1.65, times the classical error at three levels in single precision). Our scheme for `4x4x4_48_accurate_262` is equivalent to DPS's by relabelling and by scalings by powers of two, which are exact in floating point, so the first difference lies entirely in the programs. It is not their length. DPS's programs have the longer chains of roundings, up to 22 through one level against 18 for ours, and the probes' direct sums have 37 and 39 yet beat our programs (Table 8). Nor is the second difference the direct sums' multiplications by 3/4: in the model below they change the result by under 1%.

A rounding model. Table 8 models one level of each program to first order. Every addition, and every scaling by a constant that is not a power of two, rounds, with an error proportional to the exact value rounded, and the errors are independent. The inputs are independent with unit variance, so the products have covariance $(L_i \cdot L_{i'})(R_i \cdot R_{i'})$ and the size of every intermediate value, cancellation included, is an exact second moment. Propagating each rounding to the outputs gives the error each pass adds to a level.

- For `4x4x4_48_accurate_262`, DPS's programs and ours add about the same error in the L and R passes (5.74 against 5.61) but not in the P pass (3.09 against 8.22). The sixteen products that feed an output are each $1/\sqrt{2}$ of its size and would sum to $2\sqrt{2}$ times its size

if they did not cancel. DPS's P pass keeps its partial sums small: each output collects 15 roundings whose RMS size is 0.80 of the output's. Ours lets them grow: each output collects 36 roundings, since our program shares more partial sums between outputs, of RMS size 1.39.

- At one level the leaf products' own errors dominate, and the two programs measure the same. Every further level amplifies those by the same factor and adds a new pass error, so the gap grows with depth (Table 7).
- For the four programs of Section 5.2 whose schemes amplify leaf errors equally ($2\sqrt{2}$ for the worst output), the model's worst-output error orders the measured errors exactly, at three levels and at five.
- `4x4x4_48_accurate`, `4x4x4_48_rational` and `4x4x4_48_201` amplify leaf errors more (3.16, 4 and 9.53). `4x4x4_48_accurate`'s own programs add no more error than `4x4x4_48_accurate_262`'s (7.12 against 7.22); its larger measured error comes from its scheme's larger amplification. The same trade-off explains the crossover of Section 5.4: the probes and our programs for `4x4x4_48_accurate_262` amplify less than `4x4x4_48_accurate` but add more error per level (10.75–12.51 against 7.12), so they beat `4x4x4_48_accurate` while the leaves' own errors dominate and fall behind it at depth.

What the optimizer reaches. To see whether more search would have given us DPS's programs, we swept PLinOpt's `optimizer` over both schemes and all three matrices: 1000 and 5000 random loops instead of 100, its exhaustive nullspace permutations (`-N`), its factoring method (`-A`), and for P both direct optimization and the transposition pipeline. Its exhaustive CSE search (`-E`) did not finish within 120 s on any matrix. The best complete sets have 348 operations for DPS's scheme and 409 for ours, against DPS's 262 and our 445. So the 262-operation programs did not come from this search alone, and the commit does not say how they were made. The sweep did show where our P pass went wrong: every P program optimized directly, rather than through the transposition pipeline, has DPS's structure, 15 roundings per output of RMS size 0.81–0.88, and adds 3.15–3.40 against their 3.09 in the model (Table 8, last two rows). Our L and R programs still add more than DPS's (7.05–7.39 against 5.74), so the best sets model at 8.95 and 9.96 on the worst output, between our 12.51 and DPS's 7.22. We have not measured these programs.

Recommendation. Wherever `4x4x4_48_accurate` is used, `4x4x4_48_accurate_262` is better on both counts: 22 fewer operations and about a quarter less error, in the same orbit and with the same γ_2. With the programs of Section 8.4 the saving is 24 operations.

Limitations.

- The measurements come from one machine and one implementation style: fused passes over a vendor GEMM, with large leaves. The inputs are i.i.d. uniform or normal matrices, the error is the max-norm error relative to $\max|C|$, and there are at most three levels at $n = 4096$ and 8192 and five at $n = 1024$. Structured or badly scaled inputs may rank the schemes differently.
- The continuous search gives no proof of global optimality. Only the dyadic orthogonal moves are enumerated exhaustively; non-orthogonal dyadic isotropies, such as shears and scalings by two, are not.

Outlook. The natural next objective combines the growth factors with Q_0 and the operation count over dyadic points, or uses the measured error of deep recursion directly; `gamma48` makes such searches cheap. Section 8 carries the rounding model into a program search, finds the scheme's amplification of the leaf error at its floor, lowers the deep-recursion error by a third with double-precision passes, and takes two operations off DPS's programs. Lower precisions (bf16, fp16), where every level's error is a larger share of the result, are where an accurate scheme matters most.

8 Beyond the published programs

The model of Section 7 points to two ways to a realized algorithm more accurate than `4x4x4_48_accurate_262` with DPS's programs: programs that add less rounding error, and schemes that amplify the leaf products' error less. We tried both; then the one lever that remained. The last subsection turns to the operation count.

8.1 Programs found with the model

`slpopt` (Rust) ports the model, exact to every printed digit of Table 8, and searches programs of one matrix with the other two fixed: programs are DAGs of sums and scalings, the moves keep every output's linear form (rebuilding a sum tree at random or by smallest partial sums, rotating, sharing and unsharing a sum, creating a common subexpression across trees), simulated annealing drives them, and every accepted move is checked against the matrix. PLinOpt's `optimizer`, which we made 7–10 times faster for this purpose (Section 9), supplies starting programs by the thousand.

The headroom is small. Of 3000 random `optimizer` programs per matrix, scored with the other two matrices at DPS's programs, the best R models at 6.56 against DPS's 7.22, L gives one program (7.27, no ties to break) and no P beats DPS's; the optimizer's kernel and LU methods, annealing from DPS's programs, and a greedy construction that applies the model-best subexpression at each step do no better. The best set we assembled, DPS's L, the best sampled R after annealing, and a P built by pairing the smallest partial sums first, models at 6.47 (L+R 5.52, P 2.97; 539 operations; SLPchecker passes all three). Measured with the protocol of Section 5 (the rows "model-searched programs" of Tables 3, 7 and 8), its error is 1.001, 1.007 and 1.014 of `4x4x4_48_accurate_262`'s at one to three levels, within the seed-to-seed spread, and 0.95 at four and five levels, at the same speed. Ten percent less pass error is five percent at depth and nothing with large leaves: the error that dominates is the leaf products' own, amplified by the scheme.

8.2 The amplification of the leaf error is at its minimum

In the model, one level amplifies the leaf products' errors into output j by

$$\mathrm{amp}_j = \tfrac{1}{2}\Big(\sum_i p_{j,i}^2 \,\|L_i\|_2^2\,\|R_i\|_2^2\Big)^{1/2}, \tag{12}$$

the column "amp." of Table 8; `4x4x4_48_accurate_262` has $2\sqrt{2}$ for every output, `4x4x4_48_accurate` 3.16. It is the root-sum-square counterpart of $\gamma_{\infty,2}$'s per-output sum $\sum_i |p_{j,i}|\,\|L_i\|_2\|R_i\|_2$: the growth factor bounds the error with every rounding aligned, the model adds independent roundings in quadrature.

That puts the amplification on the probabilistic side of rounding-error analysis rather than the worst-case side the growth factors belong to. The classical bounds for fast multiplication – Brent's for Strassen [4], Bini and Lotti's for bilinear algorithms [3], Higham's [15], the normwise stability

of Demmel, Dumitriu, Holtz and Kleinberg [8], and the coefficient-sum constants of Ballard et al. [2] that DPS's $\gamma_{p,q}$ refine – all take every rounding at its worst and aligned, which is why they bound `4x4x4_48_accurate` and `4x4x4_48_accurate_262` alike. Higham and Mary [16] showed that rounding errors modelled as independent zero-mean random variables give bounds that grow like $\sqrt{n}$ instead of n and track measured errors far better, and Ipsen and Zhou [17] did the same for inner products with explicit distributions. The amplification is that model applied to the structure of a scheme: the r leaf products carry independent errors proportional to $\|L_i\|_2\|R_i\|_2$, and the P pass adds them in quadrature, so an output's error grows as the root-sum-square of (12), not the sum. It is a model, not a bound, and we treat it as one: the measurements of Section 5, against the exact product, are what decide between it and $\gamma_{\infty,2}$, and they follow the amplification where the two disagree.

For `4x4x4_48_accurate_262` each output collects sixteen products with $|p_{j,i}| = 1$ and $\|L_i\|_2\|R_i\|_2 = \sqrt{2}$, so the sum is $16\sqrt{2}$ and the root-sum-square $\frac{1}{2}\sqrt{16 \cdot 2} = 2\sqrt{2}$. We looked for less in every way we could:

- Over the 648 dyadic orthogonal points of Section 4.3, each scored with greedy programs, amp takes five values: $2\sqrt{2}$ at 342 points, `4x4x4_48_accurate_262`'s among them, then 3.16, 3.32, 3.46 and 3.74. The realized pass error ranges from 6.7 to 8.6, `4x4x4_48_accurate_262`'s point at 6.96 and the lowest, 6.73, in probe B's family.

- Simulated annealing over non-orthogonal dyadic isotropies (shears by ± 1, $\pm\frac{1}{2}$, ± 2, row swaps, scalings by $2^{\pm 1}$) never leaves $2\sqrt{2}$, and BFGS over all 48 parameters of the orbit from twelve random starts returns to $2\sqrt{2}$ at orthogonal points with all sixteen outputs equal. The orbit's $\gamma_{\infty,2}$ minimum, 21.41, has amp = 3.14: it minimizes the sum (21.41 against 22.63) but not the root-sum-square (3.14 against 2.83); a smaller growth factor is not a smaller amplification.

- Over the whole variety of rank-48 decompositions of the $4 \times 4 \times 4$ tensor, minimizing a soft maximum of the amp_j with the Brent equations as a penalty raised in stages (L-BFGS on all 2304 coefficients), every start returns to $2\sqrt{2}$: sixteen starts from `4x4x4_48_accurate_262` perturbed by up to the size of its coefficients land on generic points of its orthogonal sub-orbit ($\gamma_2 = 154.51$, dense irrational coefficients), and starts from Li, Wang and Hu's $s(1/2)$, whose own orbit bottoms at 3.00, leave that orbit for $2\sqrt{2}$ too.

- The other known classes, minimized over their orbits: $s(1/4)$, the DPS class, 2.83; $s(1/2)$ 3.00; $s(1)$ 3.54; $s(2)$ 4.50; $s(4)$ 6.01; $s(8)$ 8.25; Agarwal's scheme 5.80.

The minimum set is the orthogonal sub-orbit on which γ_2 is minimal, which is where DPS looked first. This is not a proof that $2\sqrt{2}$ is the floor for rank 48, but it is every basin we have a foothold in, and it closes the scheme as a lever for this measurement: whatever the program, one level of a known rank-48 scheme multiplies the leaf error by at least $2\sqrt{2}$.

8.3 Double-precision passes

The remaining lever is the passes' arithmetic. With `FMM_F64PASS=1` the library's 4×4 passes on the single-precision path read and write `float` blocks but add in `double` (a third kernel set, generated like the others), so a pass rounds once per element it stores, at the GEMM's interface, instead of at every addition; the GEMMs and the plans are unchanged. The interface roundings are amplified like the leaf error, so the pass term cannot vanish: one rounding per stored element gives, in the model, $L{+}R$ 4.0 and P 1.0, a worst output of 4.1 against DPS's programs' 7.2.

Measured with DPS's programs both ways (the rows "f64 passes" of Tables 3 and 7, and the dashed line of Figure 2), the error with the double-precision passes is 0.99, 1.02 and 0.98 of the single-precision passes' at one to three levels on $n = 4096$ and 8192, a tie where the leaves' error dominates, and 0.97, 0.96, 0.85, 0.72 and 0.64 at one to five levels on $n = 1024$: 6.25 times the classical error at five levels instead of 9.79. The passes are memory-bound, so the double arithmetic is nearly free: 1.000–1.017 of the time on the plans at 4096 and 8192 and on one level at 16384, and 1.057 on the three-level pipelined plan at 16384, where the passes are the largest share. This is the first change that lowers the deep-recursion line, and the model's floor accounts for its size. The library leaves the switch off by default; the natural rule is to turn it on with the accuracy budget, for plans of three or more fast levels.

8.4 Two operations fewer

We ran Patel's large-neighbourhood search [21], which moves through the sets of values a program computes, in our Rust port [24], starting from DPS's programs, with twelve seeds of 120 s per matrix. L stays at 72 operations and P at 112 on every seed, and R goes from 78 to 76 on five seeds and to 77 on the other seven. Another 72 seeds on R give the same two values, 76 on 23 of them. The start matters: from the matrices alone, 300 s on two seeds, the same search reaches 72 and 112 but stops at 93 on R. So `4x4x4_48_accurate_262`'s matrices cost $72 + 76 + 112 = 260$ operations, and the leading term of Section 1 falls from $9 + 3/16$ to $9 + 1/8$. PLinOpt and the current version of [13] still give 262. The 260 is an upper bound from a randomized search, and we have no proof that it is minimal.

The search counts operations, not roundings. The first set, DPS's L and P with one of those 76-operation programs for R, models at 7.59 against DPS's 7.22 and measures about 3% worse at four and five levels on $n = 1024$. So we took the 28 distinct 76-operation programs of the 84 seeds and polished each with `slpopt` (Section 8.1) under an operation budget, which rejects any move that takes a program above its count; L and P are polished too, at 72 and 112. The best set, `accurate_260`, models at 7.12 (L+R 5.67, P 3.02). It has 64 additions and 8 scalings for L, 66 and 10 for R, and 112 and none for P. Its 18 scalings are all halvings and it has no negations, so the count is the same whether or not negations are charged. DPS's programs polished the same way at their own counts reach 7.04, so the lower model value comes from the polishing and not from the two operations.

`accurate_260` passes `SLPchecker` against `4x4x4_48_accurate_262`'s published matrices and our exact check against the matrix multiplication tensor. The matrices being the same, so are the growth factors, Q_0 and the amplification.

Measured with the protocol of Section 5 on the same inputs as DPS's programs (the rows `accurate_260` of Tables 3, 7 and 8), its error is 1.006, 1.021 and 1.008 of theirs at one to three levels on $n = 4096$ and 8192 (36 pairs per level, standard errors about 0.008), and 1.021, 0.989, 0.999, 0.964 and 0.953 at one to five levels on $n = 1024$ (20 pairs per level). At the one or two levels where the library's plans run it is no more accurate; the 2% at two levels on the large sizes is 2.4 standard errors. It is 4–5% lower at four and five levels on $n = 1024$, where the error is 3.6 and 9.2 times the classical error in single precision and no plan goes. The model does not separate programs this close: `accurate_260` has 1.4% less modelled pass error than DPS's programs and the model-searched set 10%, yet at five levels they measure 9.20 and 9.25 against 9.79.

The two operations save no measurable time. Over nine alternating rounds on an idle machine, with the order swapped each round, the eight settings of Table 6 give mean ratios of 0.988–1.007, and 0.999 ± 0.002 overall. The library keeps DPS's programs, and `accurate_260` stands as an operation count.

9 Data and code

Everything is in the public repository https://github.com/gsidebottom/logic; paths below are relative to it.

- The search: `src/bin/gamma48.rs` (the continuous search) and `matmul/dps48/gamma/`, which holds the role assignments and the isotropy action (`orbit.py`), the earlier Python search (`search_orth.py`, `search_gl.py`), the exact constructions (`exact.py`), the exhaustive dyadic enumeration (`dyadic_orth.py`) and the equivalence test (`equiv.py`, `equiv262.py`).
- Growth factors: `matmul/fastmm/growth.py`, checked against PLinOpt's `growthfactor`.
- Other classes: `matmul/dps48/lwh_family.py` and `agarwal_replacement.py`, which rebuild the Li–Wang–Hu members and Agarwal's scheme from the papers.
- Measurements: `matmul/fastmm` (`bench_fmm.c`, `bench_impl.h`, `gen_kernels.py`). The builds are `build/bench_fmm_acc262` for the 262-operation programs and `build/bench_fmm_acc284` for `4x4x4_48_accurate`'s. The runs labelled `bench_fmm` in the result files used `4x4x4_48_accurate`'s programs, the library's default until it switched to `4x4x4_48_accurate_262` after these measurements. The `2x2x2_7_Winograd` rows ran the library's 2×2 level with Strassen–Winograd's programs, its default until November 2, 2026; since then the level's default programs are the dyadic accurate point of [9] (their Eq. 38), and `gen_kernels.py --sw winograd` rebuilds the kernels these rows used. `FMM_EXACT`, `FMM_NORMAL` and `FMM_SEED` select the reference, the distribution and the instance, and `FMM_REFCACHE` shares references between builds. The probes' programs are in `matmul/dps48/gamma/slp/`. Raw results are in `matmul/fastmm/results/` (`accuracy_campaign_2026-10-25.txt`, `accuracy_deep_2026-10-25.txt`, `gamma_search_2026-10-25.txt`, `gamma_classes_2026-10-25.txt`), and `matmul/fastmm/paper_acc262.py` turns them into this paper's tables and figures.
- The rounding model: `matmul/dps48/gamma/slp_errmodel.py`, with its output in `matmul/fastmm/results/slp_errmodel_2026-10-25.txt`; `paper_acc262.py--model` writes Table 8.
- Section 8: `src/bin/slpopt.rs` (the model, the program search, `realized` scoring); the program sets `matmul/dps48/gamma/slp/opt1_{L,R,P}.slp`; the amplification floor by `matmul/dps48/gamma/dyadic_points.py`, `ampwalk.py`, `ampmin.py`, `ampvariety.py` and `inspect_point.py`, with the other classes' schemes in `matmul/dps48/classes/` and results in `matmul/fastmm/results/` (`dyadic_points_realized_`, `amp_floor_`, `amp_classes_` and `amp_variety_2026-10-26.txt`); the double-precision passes in `matmul/fastmm/gen_kernels.py`, `fmm.c` and `fmm_impl.h` (`FMM_F64PASS`); the campaigns `campaign_var.sh`, `campaign_f64pass.sh` and `summarize_var.py`, with their logs `accuracy_opt1_`, `accuracy_deep_opt1_`, `accuracy_f64pass_`, `accuracy_deep_f64pass_` and `speed_*_2026-10-26.txt`. The faster PLinOpt optimizer (one generator per thread with a seed option, no transposed copy per subexpression, incremental pair bookkeeping, a word-size prime field with rational reconstruction, 7–10 times faster on these matrices) is the branch `optimizer-speed` of https://github.com/gsidebottom/plinopt.
- Section 8.4: the programs `matmul/dps48/cand48/accurate_260n_{L,R,P}.slp`, identical to `matmul/dps48/gamma/slp/a260n_{L,R,P}.slp`, from which the build `build/bench_fmm_var_a260n` is generated; the first 76-operation R program `matmul/dps48/cand48/accurate_`

260_R.slp; the search src/bin/lns48.rs, driven by seeded262.py, seededR.py and polish76.py in matmul/dps48/rfc2610/, with their output in results_td_2026-10-31.txt there; the budget in slpopt search --maxops; matmul/dps48/signnorm48.py, which removes the negations; the exact check matmul/dps48/slp48check.py. The logs are in matmul/fastmm/results/: accuracy_a260n_ and accuracy_deep_a260n_2026-11-01.txt, accuracy_a260_ and accuracy_deep_a260_2026-10-31.txt for the first set, and speed_a260n_rerun_2026-10-31.txt.

- The optimizer sweep: matmul/dps48/gamma/optsweep.sh and optsweep_score.py; its log and scores are in matmul/fastmm/results/optimizer_sweep_2026-10-26.txt and its best programs in matmul/dps48/gamma/slp/sweep_{dps,ours}_{L,R,P}.slp.
- The schemes: PLinOpt's data/4x4x4_48_accurate_262 and data/4x4x4_48_accurate [14]; our scheme equivalent to 4x4x4_48_accurate_262 is matmul/dps48/gamma/g16s2_{L,R,P}.sms and our programs for it matmul/dps48/gamma/slp/g16s2o_{L,R,P}.slp.
- The orbit's $\gamma_{\infty,2}$ minimum (Section 4): the direct-sum programs we measured, with rounded constants, matmul/dps48/gamma/slp/g214d_{L,R,P}.slp; the search's (U, V, W) that reach it from 4x4x4_48_accurate in roles 1 and 2, the first lines of matmul/dps48/gamma/rs_table1_r1.txt and rs_table1_r2.txt (the value, the start's seed, then U, V, W row by row); and its exact form in $\mathbb{Q}(\sqrt{2})$, matmul/dps48/gamma/g214_exact_{L,R,P}.txt (one entry $a + b\sqrt{2}$ per line as row col a b) with the readable listing g214_exact.txt, both written and checked by exact_q214.py.

Acknowledgments

This work was carried out in an interactive collaboration with Claude (Anthropic; the Opus 5.5 and Fable 5.1 models), which implemented the search, measurement and verification tools and much of this text under the author's direction and review. Every claim is checked mechanically by the tools listed above; the results rest on those checks rather than on trust in either the author or the tools. The algorithm comes from Dumas, Pernet and Sedoglavic's work and was published by J.-G. Dumas in PLinOpt; our search found a scheme equivalent to it, and we learned that it was already there only after constructing ours. We thank Alexandre Sedoglavic for his comments on the first draft: the parametrization of the orbit search in Section 4, Touko Ursin's priority on the $16\sqrt{2}$ point, and the place of the orbit's $\gamma_{\infty,2}$ minimum in the abstract all come from them [22]; the account of how 4x4x4_48_accurate_262 was constructed and the numbering of the equations come from his second message [23].

References

[1] A. Agarwal, *Certified local rank and uniqueness barriers for a 48-term matrix-multiplication decomposition*, arXiv:2609.13596 (2026).

[2] G. Ballard, A. R. Benson, A. Druinsky, B. Lipshitz, O. Schwartz, *Improving the numerical stability of fast matrix multiplication*, SIAM J. Matrix Anal. Appl. 37 (2016) 1382–1418.

[3] D. Bini, G. Lotti, *Stability of fast algorithms for matrix multiplication*, Numer. Math. 36 (1980) 63–72.

[4] R. P. Brent, *Algorithms for matrix multiplication*, Technical Report STAN-CS-70-157, Stanford University (1970).

[5] A. Cayley, *Sur quelques propriétés des déterminants gauches*, J. reine angew. Math. 32 (1846), 119–123.

[6] L. van Elfrinkhof, *Eene eigenschap van de orthogonale substitutie van de vierde orde*, Handelingen van het 6e Nederlandsch Natuurkundig en Geneeskundig Congres, Delft (1897).

[7] H. F. de Groote, *On varieties of optimal algorithms for the computation of bilinear mappings I. The isotropy group of a bilinear mapping*, Theoret. Comput. Sci. 7 (1978) 1–24.

[8] J. Demmel, I. Dumitriu, O. Holtz, R. Kleinberg, *Fast matrix multiplication is stable*, Numer. Math. 106 (2007) 199–224.

[9] J.-G. Dumas, C. Pernet, A. Sedoglavic, *Strassen's algorithm is not optimally accurate*, Proc. ISSAC 2024, 254–263.

[10] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *non-complex multiplications*, arXiv:2506.13242 (2025).

[11] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A more accurate rational non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *multiplications*, arXiv:2603.18699 (v1 19 March 2026, v3 29 July 2026).

[12] J.-G. Dumas, C. Pernet, A. Sedoglavic, *Towards automated generation of fast and accurate algorithms for recursive matrix multiplication*, J. Symbolic Comput. 134 (2026) 102524.

[13] J.-G. Dumas, C. Pernet, A. Sedoglavic, J. Stapleton, P. Tichavský, *Fast matrix multiplication via recursive* $\langle 4\times4\times4 : 48\rangle$ *algorithms into practice*, arXiv:2609.12027 (v1 10 September 2026, v2 2 October 2026).

[14] J.-G. Dumas, B. Grenet, C. Pernet, A. Sedoglavic, *PLinOpt, a collection of C++ routines handling linear & bilinear programs*, https://github.com/jgdumas/plinopt; data files `4x4x4_48_accurate` (commit `fb753b5`, 23 June 2026) and `4x4x4_48_accurate_262` (commit `b441528`, 13 October 2026).

[15] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.

[16] N. J. Higham, T. Mary, *A new approach to probabilistic rounding error analysis*, SIAM J. Sci. Comput. 41 (2019), A2815–A2835.

[17] I. C. F. Ipsen, H. Zhou, *Probabilistic error analysis for inner products*, SIAM J. Matrix Anal. Appl. 41 (2020), 1726–1741.

[18] X. Li, Y. Wang, S. Hu, *Substitution and quotient of the isotropy group action*, arXiv:2607.15069 (2026).

[19] A. Novikov et al., *AlphaEvolve: a coding agent for scientific and algorithmic discovery*, arXiv:2506.13131 (2025).

[20] T. Ogita, S. M. Rump, S. Oishi, *Accurate sum and dot product*, SIAM J. Sci. Comput. 26 (2005) 1955–1988.

[21] R. Patel, *matmul-4x4-48-202*, code and programs, https://github.com/rp440/matmul-4x4-48-202 (October 2026, CC0).

[22] A. Sedoglavic, private communication (e-mail to the author), 26 October 2026: comments on the first draft of this paper; the orbit search should be restricted to the factorization of GL_4 by the orthogonal matrices for the growth factor, where a convexity result ensures uniqueness, and to $SO(4)$ in Van Elfrinkhof's parametrization for the other norms; the $\gamma_{\infty,2} = 16\sqrt{2}$ point was found by Touko Ursin in mid-September 2026 with Hadamard matrices.

[23] A. Sedoglavic, private communication (e-mail to the author), 27 October 2026: how `4x4x4_48_accurate_262` was constructed (the orbit's γ_2 minimum by numerical optimization over the quotient of GL_4 by the orthogonal group, then Ursin's exploration of Hadamard matrices for $\gamma_{\infty,2}$), and that the equations of this paper should be numbered.

[24] G. Sidebottom, *201 operations for rank-48* 4×4 *matrix multiplication over rings containing* $1/2$, preprint (2026); PLinOpt data file `4x4x4_48_201`.

[25] G. Sidebottom, *logic*, source repository, https://github.com/gsidebottom/logic (2026).